HackConn 2025 - Guía rápida de conexión de pines y librerías (Arduino)

Basado en el esquemático del badge y el firmware de referencia (RAMTEC PCB v1.0).

1. Qué incluye este documento

Este documento sirve como orientación para la comunidad que va a programar el badge. Resume: (1) pines reales (GPIO) usados por cada componente, (2) librerías Arduino requeridas, (3) cómo se comparte el bus SPI entre la pantalla ST7789 y el transceiver CC1101, y (4) recomendaciones para evitar los conflictos típicos que ya se presentaron y se resolvieron en el proyecto.

2. Mapa de pines (lo más importante)

Los pines están tomados directamente del firmware de referencia (HackConn_2025.ino).

Señal / Función	GPIO (ESP32-C3)	Componente	Notas prácticas
SPI SCK	GPIO4	LCD ST7789 y CC1101	Reloj del SPI compartido.
SPI MOSI	GPIO6	LCD ST7789 y CC1101	Datos MCU -> periférico (compartido).
SPI MISO	GPIO5	CC1101	Datos periférico -> MCU. En el header del CC1101 también aparece como MISO/GDO1.
TFT CS	GPIO7	LCD ST7789	Chip Select de la pantalla. Debe quedar en HIGH cuando se usa RF.
TFT DC	GPIO3	LCD ST7789	Data/Command para comandos/datos del display.

TFT RST	GPIO0	LCD ST7789	Reset del display. OJO: GPIO0 es pin de arranque del ESP32; no lo forces a LOW al encender.
RF CSN	GPIO10	CC1101	Chip Select del CC1101. Debe quedar en HIGH cuando se usa TFT.
LED Data	GPIO20	Cadena WS2812B (12 LEDs)	Pin de datos de los LEDs direccionables.
Buzzer	GPIO21	Buzzer (vía transistor)	En el PCB se maneja con transistor NPN (activo en HIGH).
PUSH1	GPIO1	Botón 1 (Up / Menú)	Entrada con pull-up externo. Activo en 0 (LOW). También se usa para cambiar modo en el arranque (mantener 3 s).
PUSH2	GPIO2	Botón 2 (Down)	Entrada con pull-up externo. Activo en 0 (LOW).
PUSH3	GPIO8	Botón 3 (LF)	Entrada con pull-up externo. Activo en 0 (LOW).
PUSH4	GPIO9	Botón 4 (RG)	Entrada con pull-up externo. Activo en 0 (LOW).

3. Pinout de conectores (para comparar con lo que ves en el badge)

En el PCB los módulos salen a conectores. Esto te ayuda a identificar rápido cada señal con un multímetro o visualmente.

P2 - Conector LCD ST7789 (1x8):

Pin	Señal	Comentario
1	GND	Tierra
2	3.3V	Alimentación de la pantalla
3	SPI SCK	Va al GPIO4
4	SPI MOSI	Va al GPIO6
5	LCD RST	Va al GPIO0
6	LCD DC	Va al GPIO3
7	LCD CS	Va al GPIO7
8	LCD BL	Backlight. En este PCB está amarrado a 3.3V (siempre encendida).

P3 - Conector CC1101 (2x4):

Pin	Señal	Comentario
1	GND	Tierra
2	3.3V	Alimentación del CC1101
3	NC	No conectado en este diseño
4	RF CSN	Va al GPIO10
5	SPI SCK	Va al GPIO4
6	SPI MOSI	Va al GPIO6
7	SPI MISO / GDO1	Va al GPIO5 (MISO).
8	GDO2	Conectado al pin GD02 del módulo (en el firmware actual no se usa).

4. Librerías y configuración en Arduino IDE

El firmware de referencia usa estas librerías:

- Adafruit GFX Library
- Adafruit ST7789 and ST7735 Library (usa Adafruit_ST7789)
- FastLED
- Preferences (incluida con el core ESP32 de Espressif)

Recomendación de entorno: instalar el core “ESP32 by Espressif Systems” desde Board Manager y compilar para un ESP32-C3. Si tu placa no aparece como “SuperMini”, normalmente compila bien como “ESP32C3 Dev Module”.

Nota: si usas un ESP32-C3 con USB nativo, en Tools suele ayudar tener “USB CDC On Boot: Enabled” para evitar problemas de carga/monitor serial.

5. Cómo se comparte el SPI (TFT + CC1101) y por qué se trancaba antes

La pantalla ST7789 y el CC1101 comparten el mismo SPI (SCK=GPIO4, MOSI=GPIO6). La diferencia está en el Chip Select: TFT_CS (GPIO7) y RF_CSN (GPIO10). Cuando el CS de un dispositivo está en LOW, ese dispositivo “escucha” el SPI. Si dos CS quedan en LOW a la vez, los periféricos se pisan y aparecen síntomas raros: pantallas corruptas, lecturas falsas, RX que alterna OK/FAIL, etc.

En el firmware de referencia se hace esto SIEMPRE:

- Antes de hablar con CC1101: poner TFT_CS en HIGH (deshabilitar pantalla) y luego seleccionar RF_CSN.
- Antes de dibujar en la pantalla: poner RF_CSN en HIGH (deshabilitar radio) y luego usar el display.
- Para el CC1101: esperar “READY” (MISO baja a 0) antes de empezar el SPI. Si no baja en ~3 ms, se aborta. Esto evita transacciones incompletas.
- Usar SPI.beginTransaction() / endTransaction() alrededor del CC1101.

Punto clave: el CC1101 usa la línea MISO como indicador de “READY” cuando CSN cae. Por eso, el código hace un while esperando que MISO pase a LOW; si no lo hace, suelta CSN y sale.

6. Recomendaciones para quienes van a programar su propio reto

1. Respeta el mapa de pines.
2. Cuando uses RF y pantalla en el mismo programa, crea 2 funciones tipo busForRF() y busForTFT() (como en el firmware) y llámalas antes de cada bloque.
3. No hagas tft.print()/tft.draw... dentro de una transacción SPI del CC1101 (ni dentro de rutinas de interrupción). Primero termina RF, luego dibuja.
4. Mantén RF_CSN y TFT_CS en HIGH por defecto al inicio (antes de SPI.begin).
5. GPIO0 es sensible durante el arranque. Evita que el display (o cualquier circuito externo) lo mantenga en LOW al encender.

6. Botones: son activos en 0. Si lees HIGH significa “no presionado”.
7. WS2812: si notas glitches, baja brightness y evita picos de corriente. Los LEDs trabajan a 5V, el ESP32 a 3.3V.
8. Si la RF se vuelve inestable, prueba primero con el firmware de referencia (sin cambios). Luego modifica una cosa a la vez.

7. Detalles del protocolo RF del demo (para pruebas rápidas)

El demo transmite mensajes de 9 bytes exactos, con texto ASCII: “HC:PUSH 1” hasta “HC:PUSH 4”. El receptor espera 11 bytes en la FIFO: 9 de payload + 2 bytes de status (RSSI y LQI/CRC). En modo normal, el CC1101 está configurado con CRC y AutoFlush, por eso los paquetes con CRC malo se descartan.

Modo MASTER/RX: al encender, si mantienes presionado PUSH1 durante 3 segundos, el badge cambia de modo y guarda el estado en memoria (NVS).

8. Checklist rápido de troubleshooting

- Pantalla no enciende: revisar 3.3V en P2 pin2 y GND en pin1. Revisar que BL (pin8) tenga 3.3V.
- RF no recibe: confirmar que CC1101 esté alimentado con 3.3V (P3 pin2) y que CSN esté realmente en GPIO10.
- RF alterna OK/FAIL: casi siempre es conflicto de SPI (CS mal manejado) o lectura sin esperar READY. Copia el patrón ccSelect() del demo.
- LEDs no encienden: confirmar 5V en la línea de LEDs y que el pin de datos sea GPIO20.
- Botón no responde: recordar que es activo LOW; verificar pull-up de 4.7k a 3.3V en el PCB.
- Upload difícil: activar “USB CDC On Boot” y usar un cable USB bueno (datos, no solo carga).

Autor: Emeraldo Ramos (RAMTEC) - Documento de apoyo para HackConn 2025.